

CHAPTER 19

Linear Regression

The fundamental idea behind the linear regression algorithm is that it assumes a linear relationship between the features of the dataset. As a result of the pre-defined structure that is imposed on the parameters of the model, it is also called a parametric learning algorithm. Linear regression is used to predict targets that contain real values. As we will see later in Chapter 20 on logistic regression, the linear regression model is not adequate to deal with learning problems whose targets are categorical.

The Regression Model

In linear regression, the prevailing assumption is that the target variable (i.e., the unit that we want to predict) can be modeled as a linear combination of the features.

A linear combination is simply the addition of a certain number of vectors that are scaled (or adjusted) by some arbitrary constant. A vector is a mathematical construct for representing a set of numbers.

For example, let us assume a randomly generated dataset consisting of two features and a target variable. The dataset has 50 observations (see Figure 19-1).

input variables		target variable
x1	x2	y
40	73	105
31	59	145
81	18	128
58	69	116
...	...	...
66	20	144

Figure 19-1. Sample dataset

The vectors of this dataset are

$$x1 = [40 \ 31 \ 81 \ 57 \dots 66], \quad x2 = [73 \ 59 \ 18 \ 69 \dots 20], \quad y = [105 \ 145 \ 128 \ 116 \dots 144]$$

In a linear regression model, every feature has an assigned “weight.” We can say that the weight parameterizes each feature in the dataset. The weights in the dataset are adjusted to take on values that capture the underlying relationship between the features that optimally approximate the target variable. The linear regression model is formally written as

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

where

- $\hat{y}$ (pronounced y-hat) is the approximate value of the output y that we want to predict.
- θ_i , where $i = \{1, 2, \dots, n\}$, is the weight assigned to each feature in the dataset. The notation n is the size of features of the dataset.
- θ_0 represents the “bias” term.

A Visual Representation of Linear Regression

To provide more intuition, let us draw a 2-D plot of the first feature x_1 and the target variable y of the dataset with all 50 records. We are using just one feature in this illustration because it is easier to visualize with a 2-D scatter plot (see Figure 19-2).

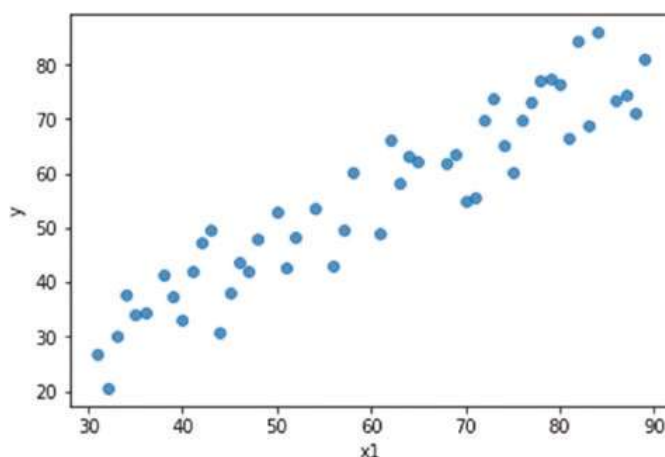


Figure 19-2. Scatter plot of x_1 (on the x axis) and y (on the y axis)

The goal of the linear model is to find a line that gives the best approximation or best fit to the data points. When found, this line will look like something in Figure 19-3. The line of best fit is known as the regression line.

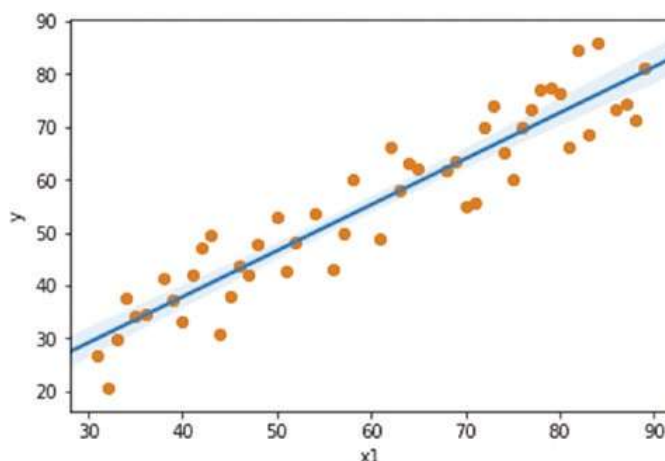


Figure 19-3. Scatter plot of x_1 (on the x axis) and y (on the y axis) with regression line

Finding the Regression Line – How Do We Optimize the Parameters of the Linear Model?

To find the regression line, we need to define the cost function, which is also called the loss function. Remember that the cost in machine learning is the error measure that the learning algorithm minimizes. We can also define the cost as the penalty when the model outputs an incorrect prediction.

In the case of the linear regression model, the cost function is defined as half the sum of the squared difference between the predicted value and the actual value. The linear regression cost function is called the ***squared error cost function*** and is written as

$$C(\theta) = \frac{1}{2} \sum (\hat{y} - y)^2$$

To put it more simply, the closer the approximate value of the target variable $\hat{y}$ is to the actual variable y , the lower our cost and the better our model.

Having defined the cost function, an optimization algorithm such as gradient descent is used to minimize the cost $C(\theta)$ by updating the weights of the linear regression model.

How Do We Interpret the Linear Regression Model?

In machine learning, the focus of linear regression differs slightly from traditional statistics. In statistics, the goal of a regression model is to understand the relationships between the features and targets by interpreting p-values, whereas in machine learning, the goal of the linear regression model is to predict the targets given new samples.

Figure 19-4 shows a regression model with a line of best fit that optimizes the squared difference between the data features and the targets. This difference is also called the residuals (shown as the purple vertical lines in Figure 19-4). What we care about in a linear regression model is to minimize the error between the predicted labels and the actual labels in the dataset.

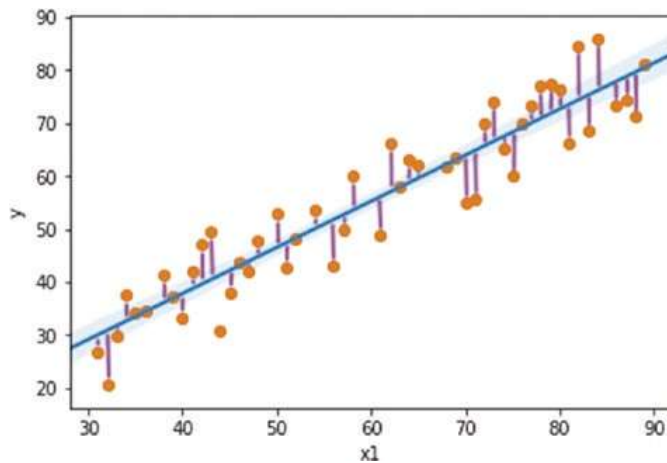


Figure 19-4. *Linear regression model showing residuals*

If all the points in Figure 19-4 entirely fall on the predicted regression line, then the error will be 0. In interpreting the regression model, we want the error measure to be as low as possible.

However, our emphasis is to obtain a low error measure when we evaluate our model on the test dataset. Recall that the test of learning is when a model can generalize to examples that it was not exposed to during training.

Linear Regression with Scikit-learn

In this example, we will implement a linear regression model with Scikit-learn. The model will predict house prices from the Boston house-prices dataset. The dataset contains 506 observations and 13 features.

We begin by importing the following packages:

`sklearn.linear_model.LinearRegression`: function that implements the `LinearRegression` model.

`sklearn.datasets`: function to load sample datasets integrated with scikit-learn for experimental and learning purposes.

`sklearn.model_selection.train_test_split`: function that partitions the dataset into train and test splits.

`sklearn.metrics.mean_squared_error`: function to load the evaluation metric for checking the performance of the model.

`math.sqrt`: imports the square-root math function. It is used later to calculate the RMSE when evaluating the model.

```
# import packages
from sklearn.linear_model import LinearRegression
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
from math import sqrt

# load dataset
data = datasets.load_boston()
# separate features and target
X = data.data
y = data.target

# split in train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, shuffle=True)

# create the model
# setting normalize to true normalizes the dataset before fitting the model
linear_reg = LinearRegression(normalize = True)

# fit the model on the training set
linear_reg.fit(X_train, y_train)
'Output': LinearRegression(copy_X=True, fit_intercept=True, n_jobs=1,
normalize=True)

# make predictions on the test set
predictions = linear_reg.predict(X_test)

# evaluate the model performance using the root mean square error metric
print("Root mean squared error (RMSE): %.2f" % sqrt(mean_squared_error(y_
test, predictions)))
'Output':
Root mean squared error (RMSE): 4.33
```

In the preceding code, using the `train_test_split()` function, the dataset is split into training and testing sets. The linear regression algorithm is applied to the training dataset to find the optimal values that parameterize the weights of the model. The model is evaluated by calling the `.predict()` function on the test set.

The error of the model is evaluated using the RMSE error metric (discussed in Chapter 14).

Adapting to Non-linearity

Although linear regression has the premise that the underlying structure of the dataset features is linear, this is, however, not the case for most datasets. It is nevertheless possible to adapt linear regression to fit or build a model for non-linear datasets. This process of adding non-linearity to linear models is called **polynomial regression**.

Polynomial regression fits a non-linear relationship to the data by adding higher-order polynomial terms of existing data features as new features in the dataset. More of this is visualized in Figure 19-5.

input variables		added higher-order polynomial terms		target variable
x_1	x_2	x_1^2	x_2^2	y
40	73	1600	5329	105
31	59	961	3136	145
81	18	6561	324	128
58	69	3364	4761	116
...	...	...	...	...
66	20	4356	400	144

50 records

Figure 19-5. Adding polynomial features to the dataset

It is important to note that from a statistical point of view, when approximating the optimal values of the weights to minimize the model, the underlying assumption of the interactions of the parameters is linear. Non-linear regression models may tend to overfit the data, but this can be mitigated by adding regularization to the model. Here is a formal example of the polynomial regression model.

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_1^2 + \theta_3 x_2 + \theta_4 x_2^2 + \dots + \theta_n x_n + \theta_n x_n^2$$

An illustration of polynomial regression is shown in Figure 19-6.

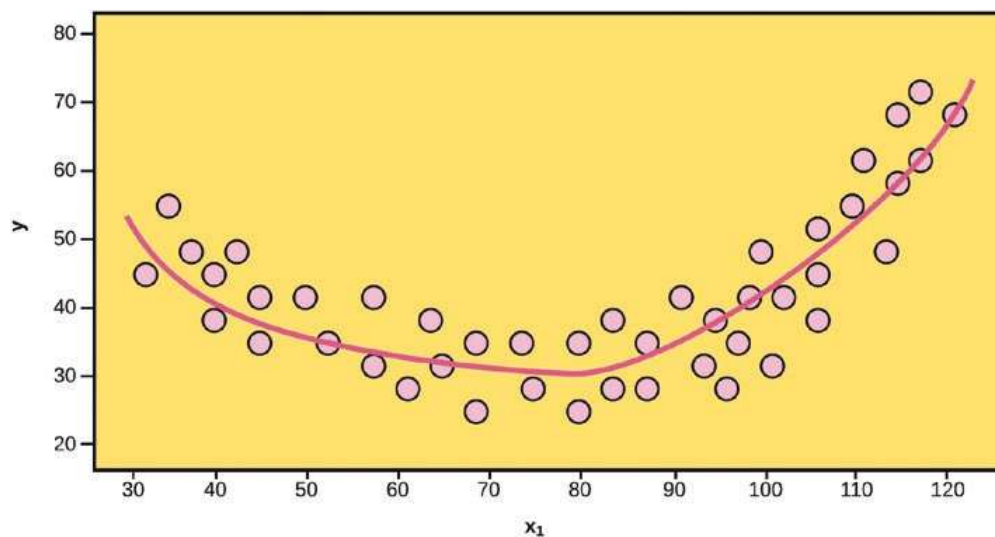


Figure 19-6. *Fitting a non-linear model with polynomial regression*

Higher-Order Linear Regression with Scikit-learn

In this example, we will create higher-order polynomials from the dataset features in hope of fitting a more flexible model that may better capture the variance in the dataset. As seen in Chapter 18, we will use the `PolynomialFeatures` method to create these higher-order polynomial and interaction features. The following code example is similar to the previous code example except where it extends the feature matrix with higher-order features.

```
# import packages
from sklearn.linear_model import LinearRegression
from sklearn import datasets
```



```

from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
from math import sqrt
from sklearn.preprocessing import PolynomialFeatures

# load dataset
data = datasets.load_boston()

# separate features and target
X = data.data
y = data.target

# create polynomial features
polynomial_features = PolynomialFeatures(2)
X_higher_order = polynomial_features.fit_transform(X)

# split in train and test sets
X_train, X_test, y_train, y_test = train_test_split(X_higher_order, y,
                                                    shuffle=True)

# create the model
# setting normalize to true normalizes the dataset before fitting the model
linear_reg = LinearRegression(normalize = True)

# fit the model on the training set
linear_reg.fit(X_train, y_train)
'Output': LinearRegression(copy_X=True, fit_intercept=True, n_jobs=None,
                           normalize=True)

# make predictions on the test set
predictions = linear_reg.predict(X_test)

# evaluate the model performance using the root mean square error metric
print("Root mean squared error (RMSE): %.2f" % sqrt(mean_squared_error(y_
test, predictions)))

'Output':
Root mean squared error (RMSE): 3.01

```

From the example, we can observe a slight improvement in the error score of the model with added higher-order features. This result is similar to what may most likely be observed in practice. It is rare to find datasets from real-world events where the features have a perfectly underlying linear structure. So adding higher-order terms is most likely to improve the model performance. But we must watch out to avoid overfitting the model.

Improving the Performance of a Linear Regression Model

The following techniques are options that can be explored to improve the performance of a linear regression model.

In the case of Bias (i.e., poor MSE on training data)

- Perform feature selection to reduce the parameter space. Feature selection is the process of eliminating variables that do not contribute to learning the prediction model. There are various automatic methods for feature selection with linear regression. A couple of them are backward selection, forward propagation, and stepwise regression. Features can also be pruned manually by systematically going through each feature in the dataset and determining its relevance to the learning problem.
- Remove features with high correlation. Correlation occurs when two predictor features are strongly dependent on one another. Empirically, highly correlated features in the datasets may hurt the model accuracy.
- Use higher-order features. A more flexible fit may better capture the variance in the dataset.
- Rescale your data before training. Unscaled features negatively affect the prediction quality of a regression model. Because of the different feature scales in multi-dimensional space, it becomes difficult for the model to find the optimal weights that capture the learning problem. As mentioned in Chapter 16, gradient descent performs better with feature scaling.

- In a rare case, we may need to collect more data. However, this is potentially costly.

In the case of variance (i.e., the MSE is good when evaluated on training data, but poor on the test data)

- A standard practice, in this case, is to apply regularization (more on this in [Chapter 21](#)) to the regression model. This can do a good job at preventing overfitting.

This chapter provides an overview on the linear regression machine learning algorithm for learning real-valued targets. Also, the chapter provided practical steps for implementing linear regression models with Scikit-learn. In the next chapter, we will examine logistic regression for learning classification problems.